

Spring Core_Maven

Exercise 2: Implementing Dependency Injection

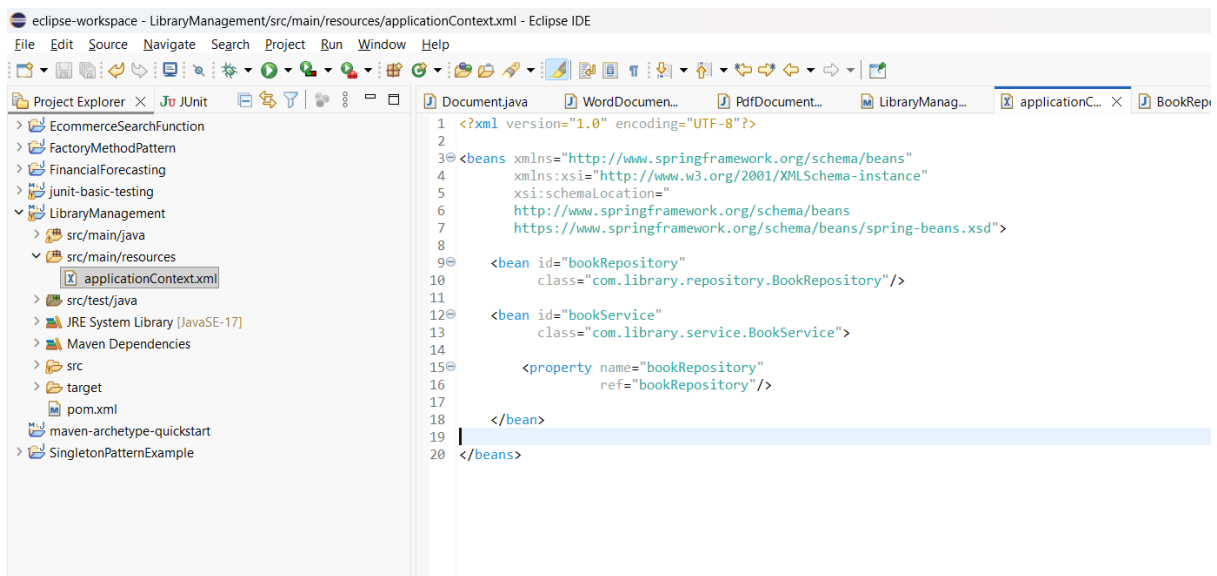
Scenario:

In the library management application, you need to manage the dependencies between the **BookService** and **BookRepository** classes using Spring's IoC and DI.

Steps:

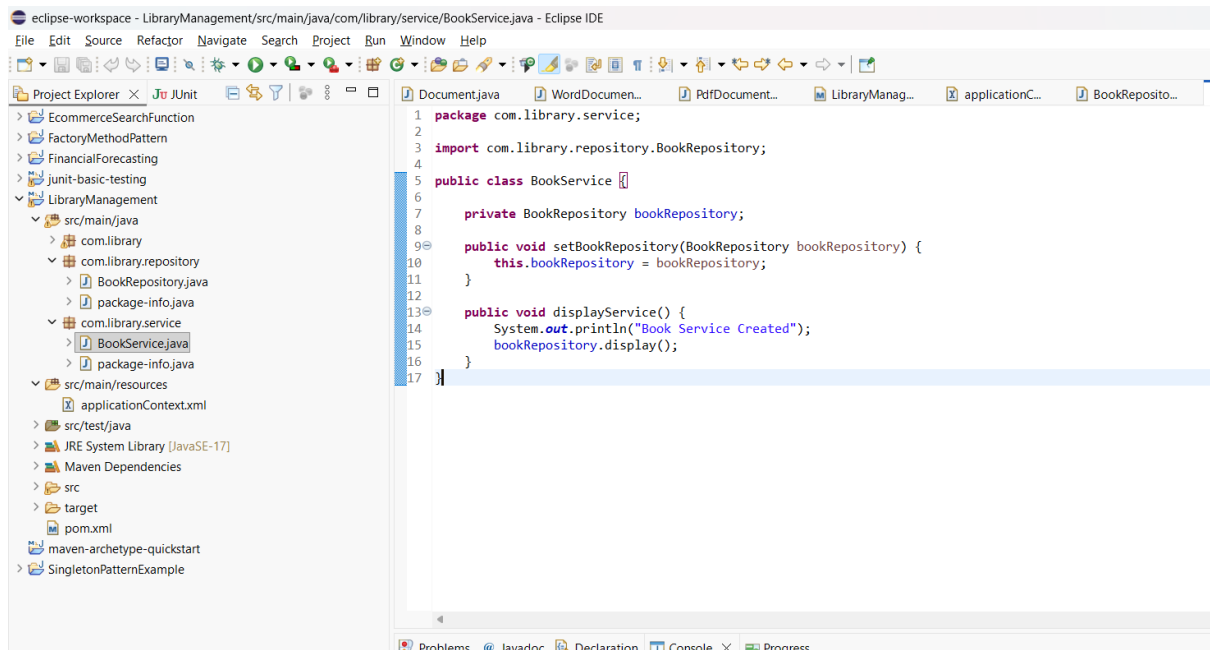
1. **Modify the XML Configuration:**
 - Update **applicationContext.xml** to wire **BookRepository** into **BookService**.
2. **Update the BookService Class:**
 - Ensure that **BookService** class has a setter method for **BookRepository**.
3. **Test the Configuration:**
 - Run the **LibraryManagementApplication** main class to verify the dependency injection.

Step 1: Modify the XML Configuration

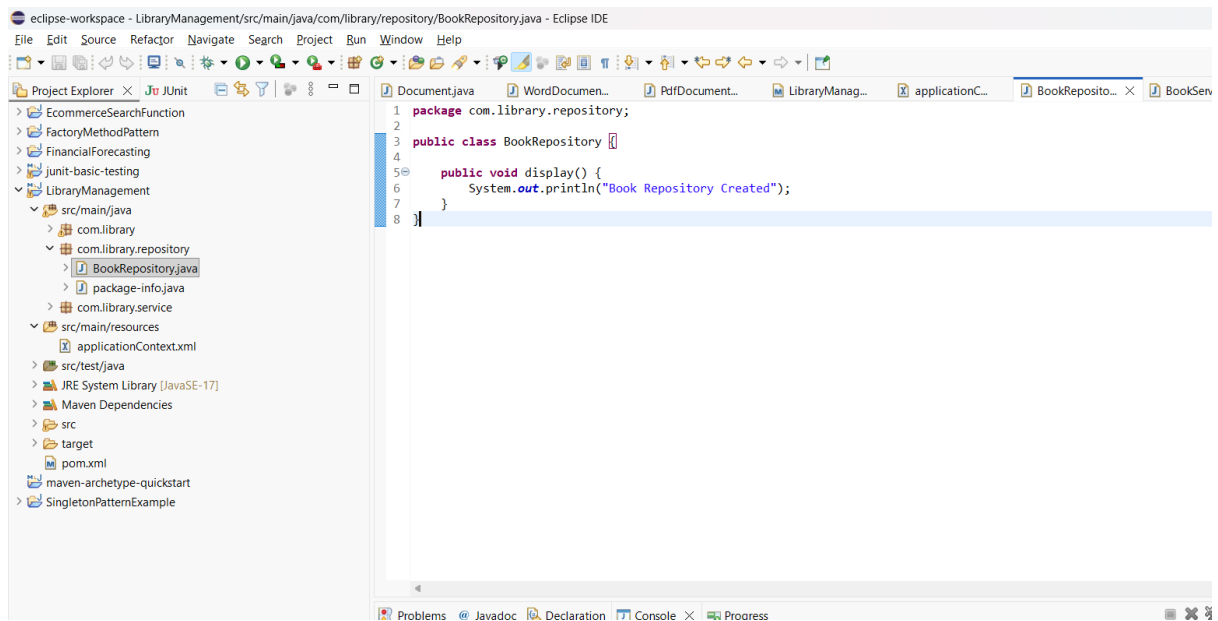


COGNIZANT DIGITAL NURTURE 5.0 – JAVA FSE

Step 2: Update the BookService Class



Step 3: Verify the BookRepository Class



COGNIZANT DIGITAL NURTURE 5.0 – JAVA FSE

Step 4: Run the LibraryApplication Class

The screenshot displays the Eclipse IDE interface. The Project Explorer on the left shows the project structure, with the `LibraryApplication.java` file selected under the `com.library` package. The main editor window shows the code for `LibraryApplication.java`, which includes the following code:

```
1 package com.library;
2
3 import org.springframework.context.ApplicationContext;
4
5 public class LibraryApplication {
6
7     public static void main(String[] args) {
8
9         ApplicationContext context =
10             new ClassPathXmlApplicationContext("applicationContext.xml");
11
12         BookService service =
13             (BookService) context.getBean("bookService");
14
15         service.displayService();
16     }
17 }
```

The bottom of the IDE shows the Console window with the following output:

```
<terminated> LibraryApplication [Java Application] C:\Users\srapy\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64.jre\bin\java.exe
Book Service Created
Book Repository Created
```